Pimpri-Chinchwad Education Trust's
Pimpri-Chinchwad College of Engineering
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU) ISO 21001:2018 Certified by TUV

DEPARTMENT OF INFORMATION TECHNOLOGY

Assignment No. 7

Subject	DBMS Lab
Assignment No.	7
Topic	No SQL: CRUD operations.
Student Name	Madhav Khobare
Roll Number	124B1F023
Date	07/04/2026
Div & Batch	SY IT A1
Guided by: -	Ms. Alpana A. Borse

Academic Year: 2025-26 Semester: IV

Aim:

To design and develop NoSQL queries using CRUD operations in MongoDB.

Objectives:

- To understand NoSQL database concepts.
- To perform CRUD operations using MongoDB.
- To use logical operators and aggregation.
- To implement joins using $lookup.
- To apply SAVE/INSERT operations.

Case Study: Airline Management System

MongoDB Queries (CRUD Operations):

---

1. CREATE (Insert Document)

Theory:
Create operation is used to insert new documents into a collection.

```
db.airline.insertMany([
  { airline_id: 1, airline_name: "Air India", country: "India" },
  { airline_id: 2, airline_name: "Indigo", country: "India" },
  { airline_id: 3, airline_name: "Emirates", country: "UAE" }
])
```

---

2. READ (Find All Data)

Theory:
Read operation retrieves documents from a collection.

```
db.airline.find()
```

Output:

```
{ airline_id: 1, airline_name: "Air India", country: "India" }
{ airline_id: 2, airline_name: "Indigo", country: "India" }
{ airline_id: 3, airline_name: "Emirates", country: "UAE" }
```

---

3. READ with Condition

```
db.airline.find({ country: "India" })
```

---

4. UPDATE Operation
Theory:

Update modifies existing documents.

```
db.airline.updateOne(
  { airline_id: 1 },
  { $set: { country: "भारत" } }
)
```

---

5. DELETE Operation

```
db.airline.deleteOne({ airline_id: 3 })
```

---

6. SAVE Method (Insert/Update)

Theory:
save() inserts or updates a document.

```
db.airline.save(
  { airline_id: 4, airline_name: "SpiceJet", country: "India" }
)
```

---

7. Logical Operator ($and)

```
db.flight.find({
  $and: [
    { source: "Mumbai" },
    { price: { $gt: 5000 } }
  ]
})
```

---

8. Logical Operator ($or)

```
db.flight.find({
  $or: [
    { destination: "Delhi" },
    { destination: "Mumbai" }
  ]
})
```

---

9. Join using $lookup

Theory:

$lookup is used to join collections in MongoDB.

```
db.booking.aggregate([
  {
    $lookup: {
      from: "flight",
      localField: "flight_id",
      foreignField: "flight_id",
      as: "flight_details"
    }
  }
])
```

---

10. Projection

```
db.airline.find({}, { airline_name: 1, _id: 0 })
```

Case Study: Civic Voice Interface System

MongoDB Queries (CRUD Operations):

---

1. CREATE

```
db.user.insertMany([
  { user_id: 1, name: "Rahul Sharma", state: "Maharashtra" },
  { user_id: 2, name: "Neha Verma", state: "Delhi" }
])
```

---

2. READ

```
db.user.find()
```

---

3. READ with Condition

```
db.user.find({ state: "Maharashtra" })
```

---

4. UPDATE

```
db.user.updateOne(
  { user_id: 2 },
  { $set: { state: "New Delhi" } }
)
```

---

5. DELETE

```
db.user.deleteOne({ user_id: 1 })
```

---

6. SAVE Method

```
db.user.save(
  { user_id: 3, name: "Amit Patil", state: "Maharashtra" }
)
```

---

7. Logical Operator ($and)

```
db.complaint.find({
  $and: [
    { state: "Maharashtra" },
    { status: "Pending" }
  ]
})
```

---

8. Logical Operator ($or)

```
db.complaint.find({
  $or: [
    { status: "Pending" },
    { status: "Resolved" }
  ]
})
```

---

9. Join using $lookup

```
db.complaint.aggregate([
  {
    $lookup: {
      from: "user",
      localField: "user_id",
      foreignField: "user_id",
      as: "user_details"
    }
  }
])
```

---

10. Projection

```
db.user.find({}, { name: 1, _id: 0 })
```

Conclusion:

In this assignment, NoSQL database operations were performed using MongoDB. CRUD operations such as Create, Read, Update, and Delete were implemented along with logical operators and joins using $lookup. This demonstrates the flexibility and efficiency of NoSQL databases in handling real-world applications.